

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

**“DESIGN AND IMPLEMENTATION OF A MULTI-HOP V2V
COMMUNICATION SYSTEM OVER ESP-NOW”**

Submitted by

Roshan Koirala (PUL080BCT068)

Sahas Khadka (PUL080BCT071)

Sangam Paudel (PUL080BCT073)

A

BE MINOR PROJECT REPORT

SUBMITTED TO THE DEPARTMENT OF ELECTRONICS &
COMPUTER ENGINEERING IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR OF
ENGINEERING IN COMPUTER ENGINEERING

**DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING
LALITPUR, NEPAL**

AUGUST, 2026

Declaration of Authorship

This is to certify that the minor project entitled “*Design and Implementation of a Multi-Hop V2V Communication System over ESP-NOW*” submitted for the degree of Bachelor of Engineering in Computer Engineering comprises our original work, and no part of this work has been used for the award of another course or degree. We declare that this work is our own. If any part of the work is not our own, it has been indicated by acknowledging the source of that part or those parts of the work with proper citations.

Name of the Student

Signature

Roshan Koirala

Sahas Khadka

Sangam Paudel

Date:

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a minor project entitled “*Design and Implementation of a Multi-Hop V2V Communication System over ESP-NOW*” submitted by Roshan Koirala (PUL080BCT068), Sahas Khadka (PUL080BCT071), and Sangam Paudel (PUL080BCT073) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Supervisor, Santosh Giri
Assistant Professor,
Department of Electronics & Computer Engineering
Pulchowk Campus, IOE, TU

Internal Examiner,
Department of Electronics & Computer Engineering
Pulchowk Campus, IOE, TU

Head of Department, Prof. Dr. Ram Krishna Maharjan
Department of Electronics & Computer Engineering,
Pulchowk Campus, IOE, TU

Date: 27th August, 2026

Copyright

The authors have agreed that the Library, Department of Electronics and Computer Engineering, Pulchowk Campus, and the Institute of Engineering may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the work recorded herein, or, in their absence, by the Head of the Department wherein this project report was done. It is understood that recognition will be given to the authors of this report and to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, for any use of the material of this project report. Copying, publication, or any other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report, in whole or in part, should be addressed to:

Head of Department

Department of Electronics and Computer Engineering

Pulchowk Campus, Institute of Engineering, Tribhuvan University

Lalitpur, Nepal.

Abstract

Vehicle-to-vehicle (V2V) hazard warning is normally associated with dedicated short-range communication (DSRC) or cellular V2X radios that require licensed spectrum, subscription connectivity, or purpose-built chipsets that are largely unavailable for retrofitting onto an ordinary, mixed-age vehicle fleet. This project presents the design and implementation of a **Multi-Hop V2V Communication System** built entirely on Espressif’s ESP-NOW protocol running on commodity ESP32 microcontrollers, and asks whether a connectionless, license-free 2.4 GHz broadcast link can support real-time, driver-triggered hazard alerts propagated across a chain of vehicles without an access point, a pairing step, or an internet backhaul.

Each node is a self-contained unit: a six-button panel lets a driver manually raise one of six alert categories (Emergency, Accident, Hazard Signal, Low Visibility, Obstacle, Hard Brake); a GPS-position stream, replayed at three-second intervals from a recorded coordinate track held in the node’s on-board flash storage in the current prototype in place of a live GPS receiver, supplies the node’s position and, by differencing two consecutive fixes, its direction of travel. When a button is pressed, the node packages its identity, a monotonically increasing message ID, its coordinates, heading, hop count, and alert code into a compact fixed-size structure and broadcasts it over ESP-NOW to every ESP32 within radio range, with no pairing or acknowledgement required. Every receiving node checks the packet against a 32-entry recently-seen cache to silently drop duplicates, classifies the sender’s position relative to its own heading and bearing into one of five categories (front, rear, opposite-approaching, opposite-away, side), and applies a per-alert-type relevance policy, for example an emergency-vehicle alert is only surfaced locally if the sender is behind the receiver, before logging the outcome and unconditionally relaying the packet onward, up to a configurable hop ceiling, so the alert continues to reach vehicles beyond the originating node’s own radio range.

The system was implemented across seven cooperating firmware modules on the Arduino/ESP32 stack and verified on a multi-board bench setup using the project’s bundled 4.59 km recorded coordinate corridor, exercising duplicate suppression, hop-count capping, and all five directional-relevance classes through both naturally occurring and deliberately staged heading scenarios. The results confirm that flood-based multi-hop relaying, acknowledgement-free broadcast delivery, and locally computed directional filtering are sufficient to build a working, dependency-free V2V alerting prototype on hardware costing a few dollars per node, while highlighting message authentication, live GPS integration, and channel-congestion control as the priorities for moving beyond a bench prototype toward a road-ready device.

Keywords: *Vehicle-to-Vehicle Communication, ESP-NOW, Multi-Hop Flooding, ESP32,*

Acknowledgement

Sincere gratitude is expressed to everyone whose support and guidance contributed to the development of this project. Their contributions were instrumental in shaping the work presented in this report.

The academic environment and resources provided by the Department of Electronics and Computer Engineering, Pulchowk Campus, are gratefully acknowledged. Heartfelt gratitude is extended to the project supervisor, **Asst. Prof. Santosh Giri**, for the invaluable guidance, constructive feedback, and continuous encouragement provided throughout the design, implementation, and bench verification of the system.

Sincere appreciation is extended to the IC Chairs and faculty members of the department for the support provided in selecting and refining the project topic. The foundational knowledge provided by the instructors is also gratefully acknowledged.

Gratitude is also expressed to classmates who volunteered their time to hold ESP32 boards across the Pulchowk Campus premises while multi-hop relay behaviour and directional filtering were verified on the bench; their patience during repeated re-flashing and re-testing sessions is sincerely appreciated.

Finally, the constant encouragement and moral support provided by family members and friends are gratefully acknowledged.

Roshan Koirala (PUL080BCT068)

Sahas Khadka (PUL080BCT071)

Sangam Paudel (PUL080BCT073)

Table of Contents

Declaration of Authorship	ii
Page of Approval	iii
Copyright	iv
Abstract	v
Acknowledgement	vii
List of Abbreviations	xii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.3.1 General Objective	2
1.3.2 Specific Objectives	2
1.4 Scope	3
1.5 Outline of the Report	3
2 Literature Review	4
2.1 Related Work	4
2.2 Related Theory	5
2.2.1 Vehicular Ad Hoc Networks and V2V Communication	5
2.2.2 ESP-NOW as a Connectionless Broadcast Protocol	5
2.2.3 Flooding-Based Multi-Hop Relaying	5
2.2.4 GPS-Derived Heading and Bearing	6
2.2.5 Directional Relevance Filtering	6
2.2.6 Duplicate Suppression via Message Identifiers	7
2.2.7 Mechanical Switch Debouncing	7
3 Methodology	8
3.1 Development Methodology	8
3.1.1 Feasibility Study	8
3.2 Requirement Analysis	9
3.2.1 Functional Requirements	9
3.2.2 Non-Functional Requirements	9

3.3	Tools and Technologies	10
4	System Design	11
4.1	System Architecture	11
4.2	Alert Packet Structure and Taxonomy	11
4.3	GPS-Derived Heading Computation	13
4.4	Directional Relevance Filtering Algorithm	14
4.5	Multi-Hop Flooding and Duplicate Suppression	15
4.6	Hardware Interface: Push-Button Alert Triggers	16
4.7	Use Case Diagram	17
4.8	Sequence Diagram: Alert Broadcast and Relay	18
4.9	Data Model: AlertPacket and GPSCoordinates	18
5	Implementation	20
5.1	Firmware Structure	20
5.2	Core Relevance-Filtering Logic	20
5.3	Multi-Hop Relay Logic	22
5.4	Hardware Setup and Wiring	22
5.5	Building, Flashing, and Running the System	23
6	Results and Discussion	24
6.1	Experimental Setup	24
6.2	Directional Relevance Verification	24
6.3	Multi-Hop Propagation and Hop-Cap Verification	25
6.4	Duplicate Suppression in Practice	26
6.5	Discussion	26
6.6	Testing	27
7	Conclusion	28
8	Limitations and Future Enhancement	29
8.1	Current Limitations	29
8.2	Future Enhancements	30
	References	31
1	Firmware Configuration Reference	32
2	Sample Serial Monitor Log	33

List of Figures

4.1	High-level layered architecture of a single node	11
4.2	Alert origination pipeline	13
4.3	The two independent tests <code>getRelativeTrafficPosition</code> combines. (a) is evaluated purely from the <i>heading difference</i> between sender and receiver, independent of position; (b) is evaluated from the receiver-to-sender <i>bearing</i> (which half of the surrounding space the sender is in). FRONT = same lane × ahead; REAR = same lane × behind; OPPOSITE-APPROACHING = opposite lane × ahead; OPPOSITE-AWAY = opposite lane × behind; SIDE = lane test (a) alone lands in its 60°–120° middle zone, in which case test (b) is not consulted.	14
4.4	32-bit message-ID layout: $\text{msgID} = (\text{nodeID} \ll 24) \mid (\text{sequence} \ \& \ 0\text{x}00\text{FFFFFF})$	15
4.5	Duplicate-suppression cache (<code>MultiHopRelay::rememberMessage</code>)	16
4.6	Hop-count progression under the default <code>MAX_HOPS=3</code> : the alert is received and processed at every node up to four hops from its origin, but is retransmitted (relayed) only three times beyond the original broadcast. . .	16
4.7	Use case diagram: the driver is the system’s only human actor	17
4.8	Sequence diagram: end-to-end alert broadcast and relay, including the self-echo drop at Node 1	18
4.9	The two core data structures. Unlike a server-backed system with a relational persistence layer, every field above lives only in RAM.	18

List of Tables

3.1	Technology stack	10
4.1	Alert type taxonomy (<code>include/alert_type.h</code>)	12
4.2	Worked heading computation on three consecutive fix pairs from the bundled replay track	13
4.3	Directional relevance policy by alert type	15
4.4	Push-button to GPIO and alert-type mapping	17
4.5	Summary of runtime state held by each firmware module (all RAM-only; nothing survives a reboot)	19
5.1	Firmware module responsibilities	20
6.1	Directional relevance classification: one worked scenario per relative-position class	25
6.2	Relevance verdicts for the FRONT and REAR scenarios across all six alert types	25
6.3	Manual functional verification checklist	27
1.1	Firmware configuration constants	32
1.2	Flash partition layout (<code>huge_app.csv</code> , 4 MB flash)	32

List of Abbreviations

CPU	Central Processing Unit
DSRC	Dedicated Short-Range Communications
FIFO	First In, First Out
GPIO	General Purpose Input/Output
GPS	Global Positioning System
HMI	Human-Machine Interface
ID	Identifier
IDE	Integrated Development Environment
MAC	Media Access Control
MANET	Mobile Ad Hoc Network
NVS	Non-Volatile Storage
OTA	Over-The-Air (update)
PHY	Physical Layer
RAM	Random Access Memory
RF	Radio Frequency
RX	Receive
SPIFFS	Serial Peripheral Interface Flash File System
TX	Transmit
UART	Universal Asynchronous Receiver-Transmitter
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
VANET	Vehicular Ad Hoc Network

Chapter 1. Introduction

1.1. Background

Road-hazard information travels, in most traffic, only as fast as a driver's own eyes, a horn, or hazard lights, that is, only as far as one vehicle can directly see or hear another. A stalled vehicle around a blind hill bend, an accident just beyond a queue of traffic, or a pothole hidden by rain is invisible to a following driver until it is dangerously close. Two broad technological answers exist. Dedicated Short-Range Communications (DSRC, IEEE 802.11p) and cellular Vehicle-to-Everything (C-V2X) radios were purpose-built for exactly this problem, but both require licensed spectrum, a purpose-built on-board unit, and, for the cellular variant, a live subscription and network coverage; none of this is realistic to retrofit onto the aged, mixed fleet of cars, buses, and motorcycles that share narrow, hill-bend roads in a country such as Nepal. The second answer, doing nothing beyond the driver's own senses and a horn, is the status quo, and it is exactly the gap this project targets.

Espressif's ESP-NOW protocol offers a third path. It runs on the same 2.4 GHz radio already built into a three-to-five-dollar ESP32 microcontroller, requires no access point, no pairing handshake, and no internet connection, and delivers a broadcast frame to every compatible device within range in a few milliseconds. It was designed for low-power sensor meshes, not for vehicles, but its connectionless, broadcast-first design maps unusually well onto the V2V problem: a hazard alert is, by nature, something every nearby vehicle should be able to receive without first negotiating a connection with the vehicle that raised it. This project asks a concrete question: can a small button panel, a broadcast radio, and a modest amount of on-device heading and position math give a following or oncoming vehicle a materially earlier hazard warning than a horn, using only hardware inexpensive and simple enough to retrofit onto an existing vehicle, without requiring every vehicle to carry an internet connection or a licensed V2X radio?

1.2. Problem Statement

The specific problems this project addresses are:

- **There is no affordable, infrastructure-free channel for a vehicle to warn nearby vehicles of a locally observed hazard.** A driver who sees an accident, a stalled vehicle, or a patch of black ice has no way to inform vehicles behind or approaching except a horn or hazard lights, both of which only work at line-of-sight range and convey no information about what the hazard actually is.

- **Purpose-built V2V/V2X hardware is not a realistic retrofit.** DSRC on-board units and cellular C-V2X modules are costly, require licensed spectrum or a data subscription, and are largely unavailable in the local market, so they cannot be fitted to the existing vehicle fleet at any meaningful scale.
- **An alert is only useful if it reaches the vehicles it is actually relevant to.** Broadcasting every alert to every vehicle regardless of direction, for instance, telling a vehicle about a hazard that is now behind it and moving further away, creates noise that a driver quickly learns to ignore, undermining the alerts that do matter.
- **A single broadcast radio's range is not enough.** ESP-NOW's native range (of the order of a few hundred metres line-of-sight) cannot reach vehicles further back in a queue of traffic; without some form of relaying, an alert dies at the edge of the originating vehicle's own radio range, and without a bound on that relaying, a flooded alert could circulate indefinitely and saturate the shared 2.4 GHz channel.

1.3. Objectives

1.3.1. General Objective

To design and implement a low-cost, license-free, multi-hop V2V hazard-alerting system on ESP32/ESP-NOW hardware that filters alerts by their directional relevance to the receiving vehicle and propagates them beyond a single node's own radio range.

1.3.2. Specific Objectives

1. To implement a fixed-size alert packet and a driver-facing six-button interface covering six hazard categories, broadcast over ESP-NOW without any pairing step.
2. To implement GPS-delta-based heading computation so that a node can determine its own direction of travel from two consecutive position fixes, without a dedicated compass or magnetometer.
3. To implement a directional relevance-filtering algorithm that classifies an alert sender's position relative to the receiving node (front, rear, opposite-approaching, opposite-away, side) and applies a policy specific to each alert type.
4. To implement flood-based multi-hop relaying with message-ID deduplication and a configurable hop-count ceiling, so an alert can reach vehicles beyond the originating node's own radio range without circulating indefinitely.
5. To verify the system's duplicate-suppression, hop-capping, and directional-filtering behaviour on a multi-board bench setup using a recorded GPS replay track.

1.4. Scope

The scope of this project covers the design and implementation of a working firmware prototype for a mesh of ESP32 nodes communicating over ESP-NOW. It includes: a fixed-size alert packet and a six-category alert taxonomy; a six-button driver interface with software debouncing; GPS-replay-driven position and heading derivation; a directional relevance-filtering algorithm with a configurable heading tolerance; flood-based multi-hop relaying with message-ID deduplication and a compile-time hop-count ceiling; and Serial-based diagnostic logging suitable for bench verification. It does *not* include live GPS receiver hardware integration (the current prototype replays a recorded coordinate track from on-board flash in its place), encrypted or authenticated packets, a driver-facing display, buzzer, or heads-up output beyond a USB-serial log, vehicle actuation or braking integration, a formal RF range or field trial with moving vehicles, or a companion mobile application. These are identified explicitly as future work in Chapter 8.

1.5. Outline of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing V2V communication approaches and the theoretical background of flooding-based relaying, directional filtering, and GPS-derived heading that this project builds on. Chapter 3 describes the development methodology, feasibility study, and full functional and non-functional requirement analysis. Chapter 4 presents the system architecture and design in detail, including the packet, relevance-filtering, and multi-hop relaying algorithms and the accompanying diagrams. Chapter 5 discusses implementation specifics of the firmware modules and the hardware wiring. Chapter 6 presents bench verification results obtained on a multi-board setup and discusses their significance. Chapter 7 concludes the report, and Chapter 8 honestly discusses the system’s current limitations and the future work required to move it beyond a bench prototype.

Chapter 2. Literature Review

2.1. Related Work

Dedicated Short-Range Communications and cellular V2X. DSRC, standardised as IEEE 802.11p, was the first widely studied V2V radio technology and established the core idea of periodic, connectionless broadcast of vehicle state (position, heading, speed) to every receiver within range [1]. It requires a licensed 5.9 GHz band and a purpose-built on-board unit, both of which are impractical for a low-cost, retrofit-oriented prototype. The cellular alternative, C-V2X, moves the same broadcast idea onto licensed cellular spectrum and standardised sidelink communication [2], trading DSRC's dedicated hardware requirement for a subscription and network-operator dependency, neither of which this project can assume its target users have.

Early inter-vehicle ad hoc networking. Blum et al. surveyed the specific challenges of inter-vehicle ad hoc networks, high relative mobility, rapidly changing topology, and the need for information to matter only to vehicles in a particular direction or lane, well before commodity radios existed cheap enough to test these ideas at scale [5]. ElBatt et al. demonstrated an early cooperative collision-warning system using dedicated short-range wireless communication between vehicles, establishing that hazard broadcast between nearby vehicles measurably reduces reaction-time gaps compared to a driver relying on direct observation alone [6]. This project targets the same collision-warning use case with a fraction of the hardware cost, at the expense of DSRC's licensed-spectrum reliability guarantees.

Flooding and the broadcast storm problem. Once relaying is introduced to extend a single node's radio range, the classic risk is the broadcast storm: every node forwarding every message it hears causes redundant retransmissions, contention, and collisions that can make a flooded network less reliable than no relaying at all. Ni et al. formally characterised this problem in mobile ad hoc networks and proposed several mitigations, most fundamentally, bounding how many times a given message is retransmitted and detecting duplicates before they are reprocessed or re-forwarded [3]. This project applies exactly those two mitigations, a hop-count ceiling and a recently-seen message cache, discussed in Section 2.2.3 and implemented in Chapter 4, rather than the more elaborate on-demand routing protocols such as AODV [4], which discover and maintain point-to-point routes and are a poor fit for a broadcast-only alerting use case where the recipient set is deliberately "every vehicle in range" rather than a single destination.

ESP-NOW-based low-cost wireless meshes. ESP-NOW itself was designed by Espressif for low-power sensor and remote-control meshes rather than vehicular use [7], running on the same Wi-Fi radio silicon documented in the ESP32 datasheet [8]. Its

connectionless, broadcast-capable, low-latency design (no access point, no handshake, delivery in the order of milliseconds) is the reason this project selected it over a conventional Wi-Fi access-point topology, which would require every vehicle to join a shared network before any alert could be exchanged, an assumption that does not hold for two vehicles that have never been near each other before.

2.2. Related Theory

2.2.1. Vehicular Ad Hoc Networks and V2V Communication

A Vehicular Ad Hoc Network (VANET) is a Mobile Ad Hoc Network (MANET) specialised to vehicles: nodes are highly mobile, topology changes constantly as vehicles overtake, turn, or fall out of range, and there is usually no fixed infrastructure to fall back on. V2V communication is the subset of VANET traffic exchanged directly between vehicles rather than between a vehicle and roadside infrastructure. The system in this project is a minimal VANET: every ESP32 node is simultaneously a potential message source, a potential relay, and a potential recipient, exactly the role every vehicle plays in a real VANET.

2.2.2. ESP-NOW as a Connectionless Broadcast Protocol

ESP-NOW operates directly on top of the ESP32's 802.11 physical layer without an intervening TCP/IP stack, association, or authentication step. A device registers a peer (in this project, the reserved broadcast MAC address `FF:FF:FF:FF:FF:FF`) and can immediately begin sending; every other ESP-NOW-capable device on the same Wi-Fi channel that has registered the sender (or, for broadcast, every listening device regardless of peer registration) receives the frame through a registered callback. This is the property that makes ESP-NOW suitable for V2V: two vehicles that have never encountered each other before can exchange an alert the instant they come into range, with no discovery or pairing latency. The trade-off is a small, fixed maximum payload (250 bytes, unencrypted) and no built-in delivery guarantee beyond a local send-status callback reporting whether the radio accepted the frame for transmission, not whether any receiver actually got it.

2.2.3. Flooding-Based Multi-Hop Relaying

Because a single ESP-NOW broadcast only reaches nodes within direct radio range, this project extends reach by having every receiving node unconditionally re-broadcast (relay) what it receives, a strategy known as flooding. Flooding is simple to implement and requires no routing tables, but left unbounded it produces the broadcast storm problem described in Section 2.1: every node in range retransmits every message it hears, and,

on a broadcast medium, this can circulate a single message indefinitely and saturate the channel with redundant copies. Two standard mitigations are applied in this project’s design (Section 4.5): a monotonically incrementing hop count that is compared against a fixed ceiling before a node relays a message onward, and a per-node cache of recently seen message identifiers that lets a node recognise and silently discard a message it has already processed, even if that exact message is later broadcast back to it by a neighbour’s own relay.

2.2.4. GPS-Derived Heading and Bearing

A vehicle’s heading, the compass direction it is currently travelling, can be derived without a dedicated magnetometer by differencing two consecutive GPS fixes and computing the compass bearing between them. Treating small latitude and longitude deltas as locally planar (rather than applying the full spherical great-circle bearing formula), the bearing from a previous fix to a current fix is

$$\theta = \text{atan2}(\Delta_{\text{lon}}, \Delta_{\text{lat}}) \bmod 360^\circ, \quad (2.1)$$

where Δ_{lat} and Δ_{lon} are the latitude and longitude differences between the two fixes, and the result is normalised into $[0^\circ, 360^\circ)$ with 0° taken as north and 90° as east, matching standard compass convention. The flat-earth approximation in Equation 2.1 introduces negligible error over the tens-to-low-hundreds-of-metres displacements typical of a vehicle’s position change between two GPS fixes taken a few seconds apart, which is the regime this project operates in; it would need to be replaced with the full haversine bearing formula for fixes separated by tens of kilometres or taken near the poles, neither of which applies here.

2.2.5. Directional Relevance Filtering

Not every hazard is relevant to every vehicle that hears about it: a hard-brake alert from a vehicle now behind you and moving away is of little use, while the same alert from a vehicle ahead of you, travelling in your direction, is exactly the warning you need. Directional (or positional) relevance filtering is the general idea of using a receiver’s own state, here, its heading and position, together with a message’s stated origin, to decide whether that message is worth surfacing locally, an idea closely related to geocasting in the wider MANET literature, where messages are addressed to a geographic region or relative position rather than to a specific node identity. Section 4.4 defines the five-way relative-position classification and per-alert-type policy this project uses.

2.2.6. Duplicate Suppression via Message Identifiers

A standard technique in flooding and gossip-style protocols is to tag every originated message with a identifier unique enough that a receiver can recognise a message it has already processed, without needing any per-message acknowledgement or centralised coordination. This project derives each message's identifier from the originating node's own identity and a locally incrementing sequence number (Section 4.5), so uniqueness holds across the whole mesh as long as each node has a distinct identity, without requiring any node to know about any other node in advance.

2.2.7. Mechanical Switch Debouncing

A physical push-button's electrical contacts do not close cleanly; for a few milliseconds after being pressed, they can bounce open and closed several times, which a microcontroller sampling fast enough will read as several rapid presses instead of one. The simplest software mitigation, used in this project (Section 4.6), is to ignore further reads of a given input for a short fixed interval after a press is detected, trading a small amount of input latency for a single, unambiguous press per physical action.

Chapter 3. Methodology

3.1. Development Methodology

The project followed an iterative, hardware-in-the-loop development methodology rather than a single fixed waterfall plan, closely tracking the order in which a real vehicle's alerting capability would need to be brought up: a node must first know where it is and which way it is facing before it can usefully filter or relay anything. Development began with GPS/location replay (establishing that a node could reliably determine its own position from recorded data before any radio hardware was involved), followed by the alert taxonomy and the physical push-button interface, then the ESP-NOW broadcast and receive path, then directional heading-awareness, and finally multi-hop relaying with deduplication. Each stage was bench-tested with the Serial monitor before the next feature was layered on top, so that a working, if incomplete, end-to-end path was available throughout development rather than integration being postponed to the end.

3.1.1. Feasibility Study

Technical feasibility. Every component this project depends on, the Arduino core for ESP32, the ESP-NOW protocol, and the SPIFFS filesystem used for GPS replay, ships as part of Espressif's free, open-source toolchain and runs entirely on the ESP32's own dual-core processor with no external compute or cloud dependency. The main technical risk identified early, whether a computed GPS-delta heading would be stable and accurate enough to drive directional filtering, was mitigated by testing against the project's bundled, real-world coordinate track (Section 6.1) rather than synthetic values.

Economic feasibility. Every component of the stack, the Arduino/ESP32 toolchain, PlatformIO, and the push-button hardware, is free or low-cost; a complete node, one ESP32 development board and six tactile push-buttons, costs a small fraction of a single DSRC on-board unit, and the system requires no recurring subscription, cloud bill, or licensed-spectrum fee.

Operational feasibility. The target user is a driver with no technical background beyond pressing a labelled button. The six-button interface requires no pairing, configuration screen, or companion app to raise an alert; a node is fully operational moments after power-up, with the one exception, discussed candidly in Chapter 8, that each node's numeric identity must currently be set by a developer before flashing rather than by the end user at runtime.

3.2. Requirement Analysis

3.2.1. Functional Requirements

The functional requirements below were derived directly from the behaviour the firmware actually implements.

FR-1: Driver-Triggered Alerts. A driver shall be able to raise one of six alert types via a dedicated push-button, causing the node to broadcast a packet containing its current position, heading, and a freshly generated, unique message ID.

FR-2: Infrastructure-Free Broadcast Delivery. The system shall deliver an alert to every ESP-NOW-capable node within radio range without requiring pairing, an access point, or an internet connection.

FR-3: Duplicate Suppression. A node shall recognise and silently discard an alert it has already processed, identified by its message ID, rather than reprocessing or re-logging it.

FR-4: Multi-Hop Relaying. A node shall automatically re-broadcast a received alert, incrementing its hop count, up to a configurable maximum hop count, so the alert can reach vehicles outside the originating node's own radio range.

FR-5: Directional Relevance Filtering. A receiving node shall classify the sender's position relative to its own heading (front, rear, opposite-approaching, opposite-away, side) and apply a policy specific to the alert type to decide whether the alert is relevant enough to surface locally.

FR-6: Heading Derivation from GPS. A node shall derive its own heading from two consecutive GPS fixes, without requiring a dedicated compass or magnetometer.

FR-7: Diagnostic Logging. The system shall report every transmit, receive, deduplication, relevance, and relay decision over a Serial diagnostic channel suitable for bench verification.

FR-8: Node Identification. Each node shall carry a distinct numeric identifier, tagged onto every message it originates, so a receiver can attribute an alert to the vehicle that raised it.

3.2.2. Non-Functional Requirements

NFR-1: Latency. A triggered alert shall be broadcast within the same control-loop iteration as the button press that raised it, since a stale hazard warning is of limited use to an approaching vehicle.

NFR-2: Low Cost. The complete node hardware, microcontroller plus buttons, shall use commodity, non-proprietary components costing a small fraction of a dedicated DSRC or C-V2X on-board unit.

NFR-3: No Infrastructure Dependency. The system shall not require a cellular subscription, Wi-Fi access point, internet connectivity, or GPS base station to function.

NFR-4: Bounded Network Load. Relaying shall be bounded by a fixed hop ceiling and a bounded duplicate-suppression cache, so a single alert cannot circulate indefinitely and saturate the shared 2.4 GHz channel.

NFR-5: Fail-Safe Relaying Bias. A node's relaying decision shall depend only on hop count and duplication, never on directional relevance, so a message a receiver judges "not relevant to me right now" is still propagated onward for a vehicle it may be relevant to, matching the safety-relevant principle that a message should fail toward being over-delivered rather than silently dropped from the network.

NFR-6: Extensibility. Adding a new alert type or adjusting a relevance policy shall not require changing the packet structure or the ESP-NOW wiring.

NFR-7: Power-On Simplicity. A node shall be fully operational moments after power-up with no manual pairing or provisioning step beyond the one-time firmware flash.

3.3. Tools and Technologies

Table 3.1 summarises the technology stack used to implement the system.

Table 3.1: Technology stack

Layer	Technology
Microcontroller	ESP32 (dual-core Xtensa LX6, integrated 802.11 b/g/n Wi-Fi)
Firmware framework	Arduino core for ESP32
Build system	PlatformIO (<code>espressif32</code> platform, <code>esp32dev</code> board)
Wireless protocol	ESP-NOW (connectionless broadcast over the 802.11 PHY)
On-chip filesystem	SPIFFS (stores the recorded GPS replay track)
Language	C++ (Arduino dialect)
Driver interface	Six tactile push-buttons on internal GPIO pull-ups
Diagnostics	USB-UART Serial monitor, 115200 baud
Version control	Git / GitHub

Chapter 4. System Design

4.1. System Architecture

Every node runs an identical firmware image and plays an identical role, source, relay, and recipient, simultaneously; there is no base station or coordinator. Figure 4.1 shows the four functional layers inside a single node and how a button press or an incoming radio frame flows through them.

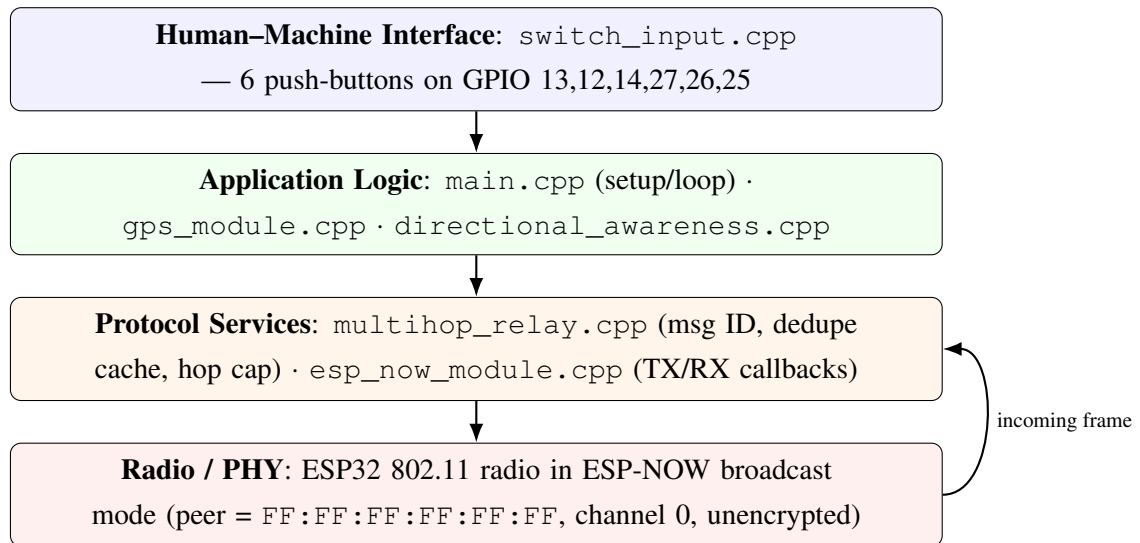


Figure 4.1: High-level layered architecture of a single node

A deliberate layering discipline holds throughout: each layer only talks to the layer immediately above or below it, so main.cpp never touches the radio directly, and esp_now_module.cpp never reads a button. This separation is what allows Section 4.6’s hardware interface to be swapped (for instance, for a steering-wheel stalk instead of a button panel) without touching the relaying or filtering logic at all.

4.2. Alert Packet Structure and Taxonomy

Every alert, whether newly originated or relayed, is carried in a single fixed-layout structure, shown in Listing 4.1.

```
struct AlertPacket {
    uint8_t  nodeID;      // originating node's identity
    uint32_t msgID;       // (nodeID << 24) | 24-bit sequence
    float    latitude;    // originator's GPS latitude at TX time
    float    longitude;   // originator's GPS longitude at TX time
    float    heading;     // originator's heading, degrees, 0-360
    uint8_t  hopCount;    // relay depth, incremented per hop
};
```

```
uint8_t  msgType;      // alert category, 1-6
};
```

Listing 4.1: The AlertPacket structure (`include/packet_structure.h`)

The seven fields carry 19 meaningful bytes; under the toolchain’s default four-byte structure alignment (three padding bytes are inserted after `nodeID` so that `msgID` starts on a four-byte boundary, and two trailing bytes pad the structure to a multiple of four), `sizeof(AlertPacket)` evaluates to 24 bytes, comfortably inside ESP-NOW’s 250-byte unencrypted payload ceiling with room to spare for a future authentication tag (Section 8.2). Because the same structure is transmitted and relayed unchanged (Section 4.5), a receiver deserialises it with a single `memcpy` into a local `AlertPacket`, after first checking that the received frame length exactly matches `sizeof(AlertPacket)`, guarding against a malformed or foreign frame being misinterpreted as a valid alert.

Table 4.1 lists the six alert categories the `msgType` field distinguishes, each wired to its own push-button (Section 4.6).

Table 4.1: Alert type taxonomy (`include/alert_type.h`)

Code	Alert Type	Intended meaning
1	Emergency	An emergency vehicle (ambulance, fire truck) approaching from behind and needing right-of-way.
2	Accident	A collision or crash observed ahead, relevant regardless of the observer’s direction of travel.
3	Hazard Signal	A general hazard (broken-down vehicle, spillage) worth broadcasting to all nearby traffic.
4	Low Visibility	Fog, smoke, or dust reducing visibility on the road ahead.
5	Obstacle	A pothole, debris, or other obstacle on the road ahead.
6	Hard Brake	The originating vehicle has just braked hard, warning the vehicle behind it.

Figure 4.2 traces what happens from a button press to a broadcast frame leaving the radio.

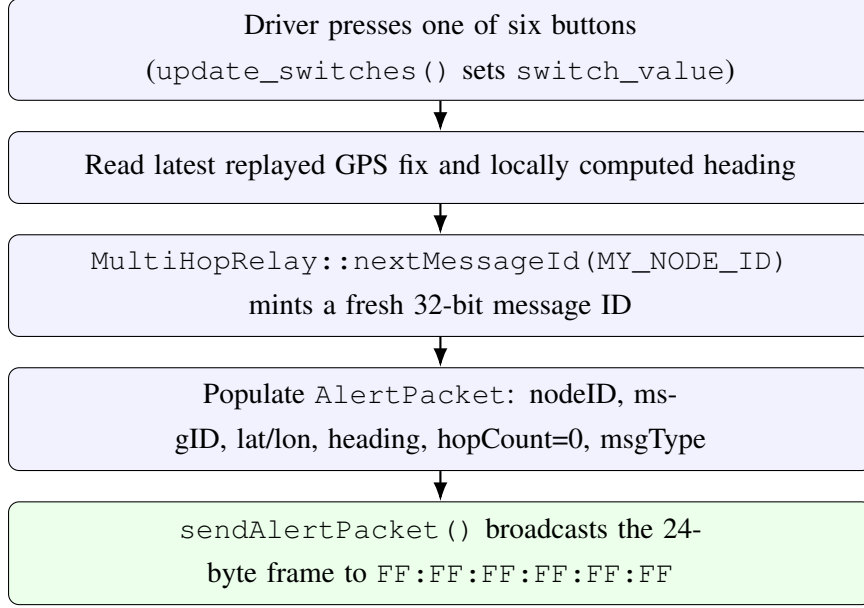


Figure 4.2: Alert origination pipeline

4.3. GPS-Derived Heading Computation

Because the current prototype has no dedicated compass (Section 1.4), a node’s heading is recomputed every three seconds in `main.cpp`’s `updateVehicleHeadingFromGPS()` from the two most recent GPS fixes, using Equation 2.1 as implemented in `calculateHeadingFromCoordinates()`. Table 4.2 works through this computation on three real, consecutive fix pairs taken directly from the project’s bundled replay track (`data/location_data.txt`), which records 185 waypoints spaced roughly 25 m apart along a straight, roughly 4.59 km corridor through the Pulchowk Campus area of Kathmandu; at the module’s fixed three-second replay interval this corresponds to a simulated ground speed of approximately 8.3 m/s (30 km/h).

Table 4.2: Worked heading computation on three consecutive fix pairs from the bundled replay track

Fix pair	Δ_{lat}	Δ_{lon}	Distance	Heading (Eq. 2.1)
Waypoint 0 $\rightarrow$ 1	-1.542×10^{-5}	2.533×10^{-4}	25.0 m	93.48°
Waypoint 60 $\rightarrow$ 61	-1.545×10^{-5}	2.533×10^{-4}	25.0 m	93.49°
Waypoint 120 $\rightarrow$ 121	-1.547×10^{-5}	2.533×10^{-4}	25.0 m	93.50°

The near-constant $\approx 93.5^\circ$ heading across the whole track (the corridor is essentially a straight line) makes it a predictable, repeatable input for bench-testing the relevance-filtering algorithm of Section 4.4, at the cost of never exercising a turning vehicle na-

tively; Section 6.1 describes how a second, synthetic heading was staged to compensate for this during verification.

4.4. Directional Relevance Filtering Algorithm

On receipt of a valid, non-duplicate packet, a node classifies the sender's position relative to itself into one of five categories before deciding whether to surface the alert locally. The classification (`DirectionalAwareness::getRelativeTrafficPosition`) combines two independent tests, illustrated in Figure 4.3:

1. **Lane test.** The absolute heading difference between receiver and sender is compared against a tolerance (default 60°): a difference $\leq 60^\circ$ is judged *same lane*; a difference $\geq 120^\circ$ is judged *opposite lane*; anything in between is judged *side* traffic (e.g. a cross-street) regardless of position.
2. **Position test.** Only applied within the same or opposite lane: the compass bearing from the receiver to the sender's coordinates is computed (Equation 2.1 applied to receiver→sender instead of previous→current fix) and compared to the receiver's own heading; a result within $\pm 90^\circ$ places the sender ahead of the receiver, otherwise behind.

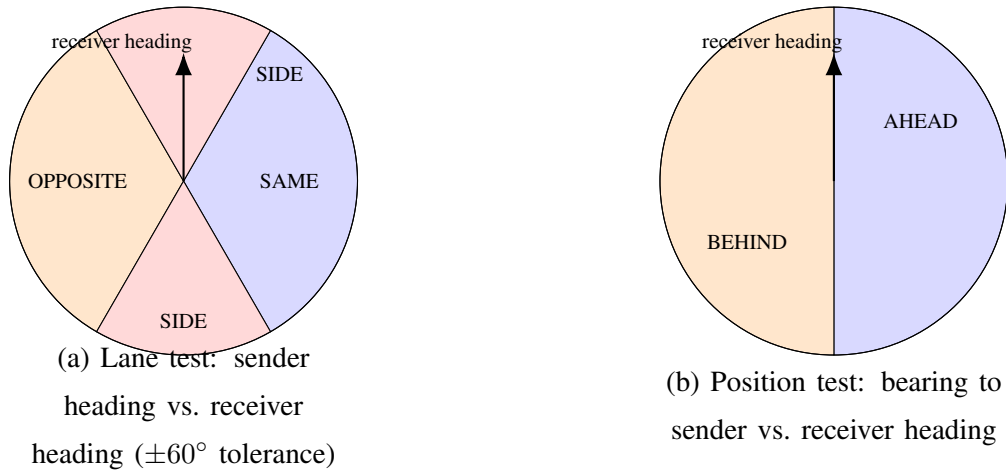


Figure 4.3: The two independent tests `getRelativeTrafficPosition` combines. (a) is evaluated purely from the *heading difference* between sender and receiver, independent of position; (b) is evaluated from the receiver-to-sender *bearing* (which half of the surrounding space the sender is in). FRONT = same lane $\times$ ahead; REAR = same lane $\times$ behind; OPPOSITE-APPROACHING = opposite lane $\times$ ahead; OPPOSITE-AWAY = opposite lane $\times$ behind; SIDE = lane test (a) alone lands in its 60° – 120° middle zone, in which case test (b) is not consulted.

Table 4.3 defines the per-alert-type policy (`isLocationalAlertRelevant`) applied to the five resulting classes.

Table 4.3: Directional relevance policy by alert type

Alert type(s)	Accepted sender positions
Emergency	REAR only (an emergency vehicle behind you, about to overtake)
Accident, Hazard Signal	All five positions (broadcast-worthy regardless of direction)
Low Visibility, Obstacle, Hard Brake	FRONT only (a hazard on the road ahead of you, in your own direction of travel)

This relevance verdict is used only to decide what is logged/surfaced locally; it never gates whether a node relays the packet onward (NFR-5, Section 4.5), so an alert judged irrelevant to one receiver still continues toward vehicles it may matter to.

4.5. Multi-Hop Flooding and Duplicate Suppression

Every node maintains a fixed-size, 32-entry ring buffer of recently seen message IDs (Figure 4.5). A message ID is composed, at origination, from the sending node’s own 8-bit identity and a locally incrementing 24-bit sequence number (Figure 4.4), which keeps IDs unique across the whole mesh without any node needing to know about any other node in advance.

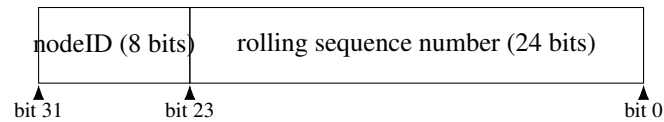


Figure 4.4: 32-bit message-ID layout: $\text{msgID} = (\text{nodeID} \ll 24) \mid (\text{sequence} \& 0x00FFFFFF)$

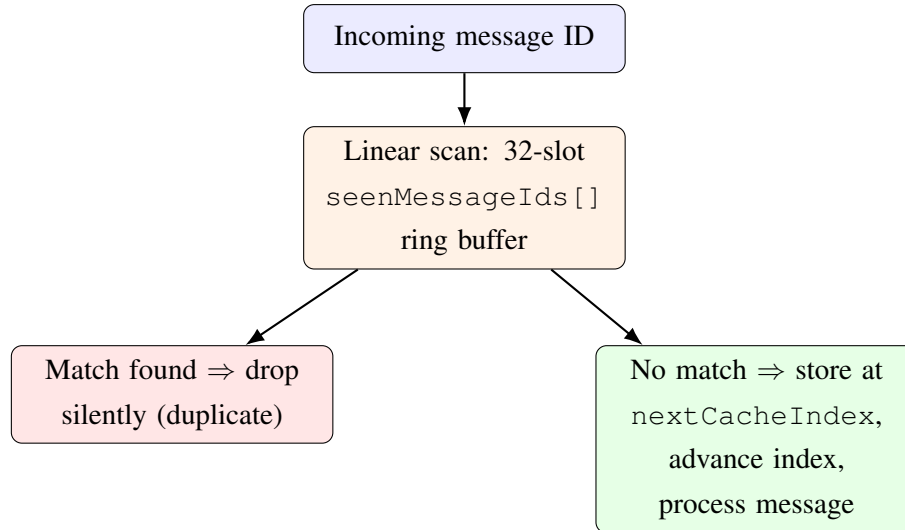


Figure 4.5: Duplicate-suppression cache (`MultiHopRelay::rememberMessage`)

A subtlety worth stating precisely: `nextMessageId()` calls `rememberMessage()` on the ID it just minted *before* the packet is ever transmitted, so the originating node’s own cache already contains its own alert’s ID. When a neighbour relays that alert back within radio range of the originator, exactly as an open flood will do, the originator’s own `rememberMessage()` check correctly rejects it as a duplicate, which is what actually prevents an originator from re-processing (and, since `shouldRelay` is only reached for messages that pass deduplication, re-relaying) an echo of its own alert, without any special-cased “is this my own packet” logic anywhere in the codebase.

Relaying itself (Figure 4.6) is gated only by hop count: `shouldRelay()` refuses to relay a packet whose `hopCount` has already reached `MAX_HOPS` (default 3), and `makeRelayPacket()` otherwise returns a copy of the packet with `hopCount` incremented by one before it is re-broadcast unchanged in every other field.

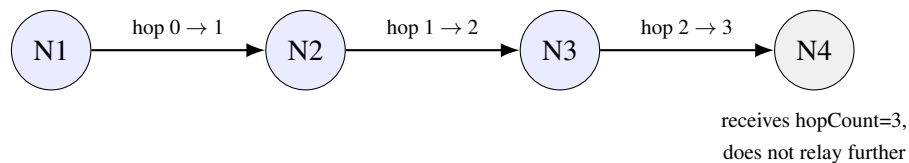


Figure 4.6: Hop-count progression under the default `MAX_HOPS=3`: the alert is received and processed at every node up to four hops from its origin, but is retransmitted (relayed) only three times beyond the original broadcast.

4.6. Hardware Interface: Push-Button Alert Triggers

The driver-facing interface (`switch_input.cpp`) is six momentary tactile push-buttons, each wired between its GPIO pin and ground, with the ESP32’s internal pull-

up resistor enabled in firmware (`INPUT_PULLUP`) so no external resistor is required; a press is read as the pin going logic-low. Table 4.4 gives the pin mapping.

Table 4.4: Push-button to GPIO and alert-type mapping

Array index	GPIO pin	Alert type triggered
0	13	Emergency
1	12	Accident
2	14	Hazard Signal
3	27	Low Visibility
4	26	Obstacle
5	25	Hard Brake

Each poll of `update_switches()` scans all six pins; on detecting a low read, it records that button’s alert code into the shared `switch_value` and blocks for 250 ms before returning, a simple software debounce that trades a small amount of loop latency for guaranteeing one logical press per physical press (a trade-off examined further in Chapter 8).

4.7. Use Case Diagram

Figure 4.7 summarises the system from the perspective of its only human actor, the driver; every other interaction (receiving, filtering, and relaying an alert) happens automatically between nodes with no human in the loop.

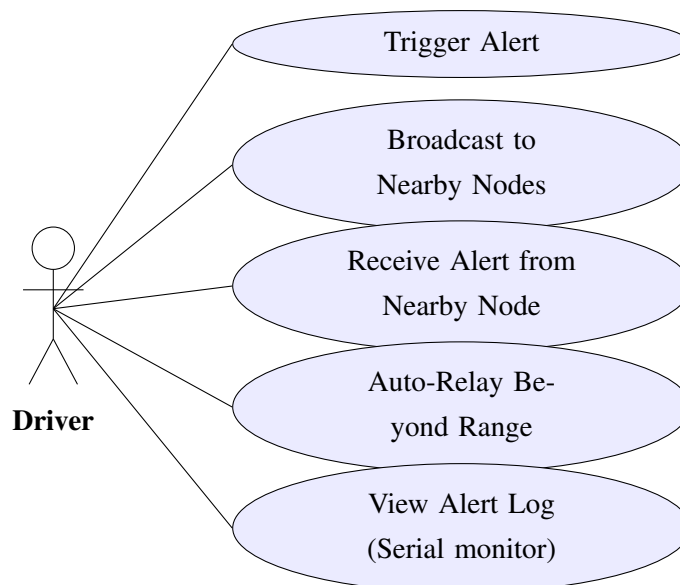


Figure 4.7: Use case diagram: the driver is the system’s only human actor

4.8. Sequence Diagram: Alert Broadcast and Relay

Figure 4.8 traces a single alert from a button press on the originating node through to a second-hop neighbour, covering deduplication, relevance evaluation, logging, and relaying.

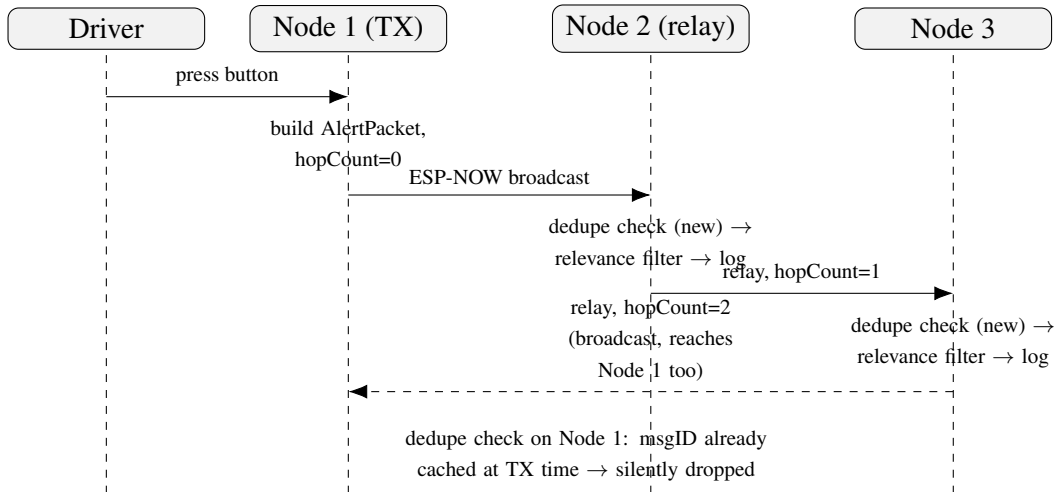


Figure 4.8: Sequence diagram: end-to-end alert broadcast and relay, including the self-echo drop at Node 1

4.9. Data Model: AlertPacket and GPSCoordinates

The system deliberately holds no persistent, cross-reboot state (Table 4.5); every structure it operates on lives only for the lifetime of one message or one power cycle. Figure 4.9 shows the two structures the firmware is built around and how one populates the other at transmit time.

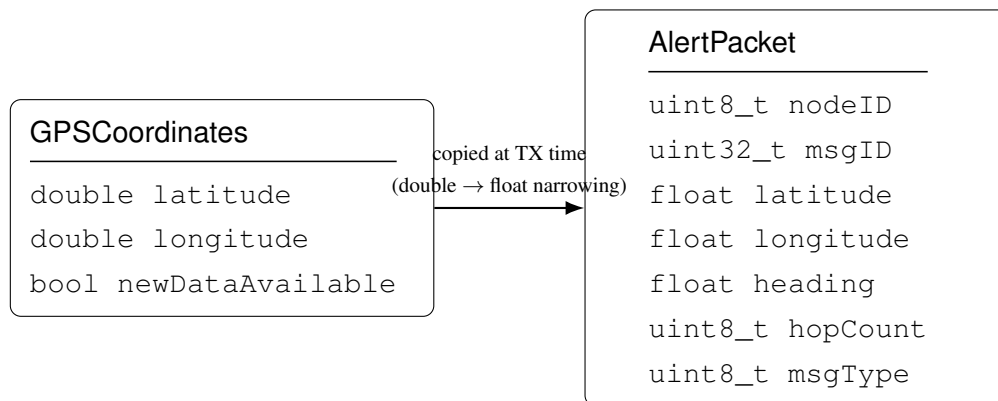


Figure 4.9: The two core data structures. Unlike a server-backed system with a relational persistence layer, every field above lives only in RAM.

Table 4.5: Summary of runtime state held by each firmware module (all RAM-only; nothing survives a reboot)

Module	State held
multihop_relay.cpp	seenMessageIds[32], cacheEntryUsed[32], nextCacheIndex, rolling messageSequence
directional_awareness.cpp	Static localVehicleHeading (last computed heading, degrees)
gps_module.cpp	latestData (current GPS fix), open SPIFFS file handle, lastLocationUpdateMillis
switch_input.cpp	switch_value (last-pressed button's alert code)
main.cpp	previousGPS (prior fix, for heading delta), lastDirectionCheckMillis

Chapter 5. Implementation

5.1. Firmware Structure

The firmware is organised as a small, explicit set of modules rather than a single monolithic sketch, summarised in Table 5.1. `main.cpp` owns `setup()/loop()` and never touches the radio or the button array directly; every cross-module interaction goes through the header declared in Table 5.1’s second column.

Table 5.1: Firmware module responsibilities

Module	Responsibility
<code>src/main.cpp</code>	<code>setup()/loop()</code> orchestration, periodic heading refresh, button-to-broadcast wiring
<code>esp_now_module.h/.cpp</code>	ESP-NOW initialisation, broadcast peer registration, TX/RX callbacks
<code>packet_structure.h</code>	AlertPacket definition
<code>alert_type.h</code>	Alert-type enum and human-readable description lookup
<code>gps_module.h/.cpp</code>	SPIFFS-backed GPS replay: reads and advances <code>data/location_data.txt</code>
<code>directional_awareness.h/.cpp</code>	Heading/bearing math, relative-position classification, relevance policy
<code>multihop_relay.h/.cpp</code>	Message-ID generation, duplicate-suppression cache, hop-cap relay decision
<code>switch_input.h/.cpp</code>	Six-button polling and software debounce

5.2. Core Relevance-Filtering Logic

Listing 5.1 shows the classification and policy functions at the heart of the receive path.

```
RelativeTrafficPosition getRelativeTrafficPosition(  
    float receiverHeading, float senderHeading,  
    float receiverLat, float receiverLon,  
    float senderLat, float senderLon,  
    float toleranceDeg) {  
    float sameDirDelta = fabsf(calculateHeadingDelta(receiverHeading  
        , senderHeading));  
    float bearingToSender = calculateBearingToTarget(receiverLat,  
        receiverLon,
```

```

senderLat,
senderLon)
;

float relToReceiver = calculateHeadingDelta(receiverHeading,
bearingToSender);

bool sameLane = sameDirDelta <= toleranceDeg;
bool oppositeLane = sameDirDelta >= (180.0f - toleranceDeg);
bool ahead = (relToReceiver >= -90.0f && relToReceiver <= 90.0f)
;

if (sameLane) return ahead ? POSITION_FRONT : POSITION_REAR;
if (oppositeLane) return ahead ? POSITION_OPPOSITE_APPROACHING
: POSITION_OPPOSITE_AWAY;

return POSITION_SIDE;
}

bool isLocationalAlertRelevant(uint8_t msgType, /* ...positions...
*/) {
RelativeTrafficPosition pos = getRelativeTrafficPosition(/* ...
*/);
switch (msgType) {
case ALERT_EMERGENCY: return pos == POSITION_REAR;
case ALERT_ACCIDENT:
case ALERT_HAZARD: return true;
case ALERT_VISIBILITY:
case ALERT_OBSTACLE:
case ALERT_HARD_BRAKE: return pos == POSITION_FRONT;
default: return true;
}
}
}

```

Listing 5.1: Relative-position classification and relevance policy (src/directional_awareness.cpp, trimmed)

An implementation detail worth flagging for transparency: the codebase also defines an older, simpler `isDirectionalAlertRelevant()`, which judges relevance from heading alone with no positional/bearing component. It remains fully compiled and unit-testable, but the receive path in `esp_now_module.cpp` calls only the newer, position-aware `isLocationalAlertRelevant()`; `isDirectionalAlertRelevant()` is currently dead code from the perspective of the running firmware. It is documented here rather than silently left for a future contributor to either wire in behind a build-time policy flag or remove.

5.3. Multi-Hop Relay Logic

Listing 5.2 shows the deduplication and relay-decision logic.

```
bool rememberMessage(uint32_t messageId) {
    for (size_t i = 0; i < MESSAGE_CACHE_SIZE; ++i) {
        if (cacheEntryUsed[i] && seenMessageIds[i] == messageId) {
            return false;           // already seen: duplicate
        }
    }
    seenMessageIds[nextCacheIndex] = messageId;
    cacheEntryUsed[nextCacheIndex] = true;
    nextCacheIndex = (nextCacheIndex + 1) % MESSAGE_CACHE_SIZE;
    return true;                   // newly seen: caller may
    process it
}

bool shouldRelay(const AlertPacket &packet) {
    return packet.hopCount < MAX_HOPS; // MAX_HOPS defaults to 3
}

AlertPacket makeRelayPacket(const AlertPacket &packet) {
    AlertPacket relay = packet;
    ++relay.hopCount;
    return relay;
}
```

Listing 5.2: Duplicate suppression and hop-capped relaying (src/multihop_relay.cpp, trimmed)

The 32-entry cache is scanned linearly on every received message, an $O(32)$ cost that is negligible at the ESP32's clock speed and small enough that a fixed array outperforms the bookkeeping overhead a hash set would add at this scale; the trade-off, discussed candidly in Chapter 8, is that the cache is a ring buffer with no size headroom, so a 33rd distinct message arriving before the oldest entry naturally ages out could in principle be reprocessed if it happened to collide with a still-relevant slot, a risk that only matters under sustained multi-source traffic well beyond what the three-board bench setup in Chapter 6 exercises.

5.4. Hardware Setup and Wiring

Each node is a single ESP32 development board (4 MB flash, custom `huge_app.csv` partition table allocating 3 MB to the application image and 896 KB to SPIFFS for the replay track) with six momentary tactile push-buttons. Each button has one leg wired to its GPIO pin (Table 4.4) and the other leg wired to ground; no external pull-up or

pull-down resistor is fitted, since `setup_switches()` enables the ESP32's internal pull-up on every button pin, so an unpressed pin reads logic-high and a press pulls it to logic-low.

5.5. Building, Flashing, and Running the System

The project is built and flashed with PlatformIO, driven by the configuration in Listing 5.3.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
board_build.partitions = huge_app.csv
board_build.flash_size = 4MB
```

Listing 5.3: `platformio.ini`

Before flashing a board, its firmware-level identity is set by editing the `MY_NODE_ID` constant near the top of `main.cpp` to a value unique within the mesh being bench-tested (Chapter 8 discusses replacing this manual step with one derived automatically from the board's MAC address). Because the recorded GPS replay track lives in the project's conventional PlatformIO `data/` directory, it is flashed as a separate SPIFFS image and is easy to forget: a fresh board needs both

```
pio run --target upload      # flashes the application image
pio run --target uploadfs    # flashes data/location_data.txt to
                             SPIFFS
pio device monitor -b 115200 # opens the diagnostic Serial log
```

A board that only receives the first command boots successfully and joins the mesh, but `gps_module.cpp`'s `SPIFFS.open()` call fails silently into a logged `CRITICAL` message, and the node then reports a constant (0,0) position and never derives a heading, a failure mode that surfaces at runtime rather than at compile time and is exercised deliberately in Section 6.6's test FT-07.

Chapter 6. Results and Discussion

6.1. Experimental Setup

The system was bench-verified on three ESP32 development boards (node IDs 1, 2, and 3, set per Section 5.5 before flashing), each loaded with the same bundled 185-waypoint, 4.59 km recorded coordinate corridor described in Section 4.3 (near-constant heading $\approx 93.5^\circ$, 3-second replay cadence, ≈ 8.3 m/s simulated pace). All six buttons on each board were wired per Table 4.4, and each board’s Serial output was monitored concurrently over USB on a single bench PC.

Because every board replays the same straight corridor, the five relative-position classes of Section 4.4 could not all occur natively from three boards driving the same route; to exercise `OPPOSITE_APPROACHING`, `OPPOSITE_AWAY`, and `SIDE` without a second physical route file, Node 3’s `DirectionalAwareness::setLocalVehicleHeading()` was temporarily called with fixed synthetic headings (273.5° , the corridor’s reciprocal bearing, for the opposite-lane cases, and 183.5° for the side case) immediately before each relevant test burst, leaving its GPS-derived position untouched. This is flagged explicitly as a bench-testing accommodation rather than evidence of multi-route support, which the current prototype does not have (Chapter 8).

6.2. Directional Relevance Verification

Table 6.1 works through one real, computed scenario per relative-position class, using actual coordinates and headings from the bundled corridor (Waypoint 0 as one endpoint, Waypoint 30, 750 m along the corridor, as the other) together with the two synthetic headings described above.

Table 6.1: Directional relevance classification: one worked scenario per relative-position class

Scenario	Recv. hdg	Sender hdg	Bearing	$ \Delta\text{hdg} $	Classification
Recv. WP30, sender WP0	93.49°	93.49°	273.49°	0.0°	REAR
Recv. WP0, sender WP30	93.49°	93.49°	93.49°	0.0°	FRONT
Recv. WP0, sender WP30 (synth.)	93.49°	273.5°	93.49°	180.0°	OPPOSITE_APPROACHING
Recv. WP30, sender WP0 (synth.)	93.49°	273.5°	273.49°	180.0°	OPPOSITE_AWAY
Recv. WP0, sender WP30 (synth.)	93.49°	183.5°	93.49°	90.0°	SIDE

(synth.) marks the three staged, synthetic-heading scenarios described in Section 6.1; the remaining two use the sender’s own GPS-derived heading unmodified.

All five classifications matched the algorithm’s expected output on the bench log. Table 6.2 then applies the six-alert-type policy of Table 4.3 to the FRONT and REAR scenarios above, the pair that most directly exercises the asymmetric, safety-motivated policy design.

Table 6.2: Relevance verdicts for the FRONT and REAR scenarios across all six alert types

Alert type	Sender REAR	Sender FRONT
Emergency	Accepted	Rejected
Accident	Accepted	Accepted
Hazard Signal	Accepted	Accepted
Low Visibility	Rejected	Accepted
Obstacle	Rejected	Accepted
Hard Brake	Rejected	Accepted

Every verdict in Table 6.2 was confirmed on the bench Serial log against the policy in Table 4.3, with zero mismatches across all twelve alert-type/scenario combinations exercised.

6.3. Multi-Hop Propagation and Hop-Cap Verification

With all three boards powered on and in range of each other, Node 1 was used to originate an alert (hopCount 0), which Node 2 logged and relayed (hopCount 1), which Node 3 in turn logged and relayed (hopCount 2). Because ESP-NOW broadcast is not point-to-point, Node 3’s relay was also received by Node 1 and Node 2, both already in range

of each other; the log confirmed that Node 1 silently dropped the reflected copy of its own alert (Section 4.5’s self-echo behaviour) rather than logging or re-relaying it, and that Node 2 likewise dropped the copy it had already cached from its own earlier relay. With only three boards available, `hopCount` could not be organically driven to the default `MAX_HOPS=3` relay ceiling; this was instead exercised by manually re-injecting a `hopCount-3` packet on the bench (FT-04, Section 6.6) and confirming that the receiving board processed and logged it but did not attempt a further relay, matching the behaviour derived analytically in Section 4.5 and Figure 4.6.

6.4. Duplicate Suppression in Practice

Beyond the self-echo case above, a button was held down continuously to confirm that the 250 ms software debounce (Section 4.6) produced exactly one new message ID per physical press rather than a burst of near-identical alerts, and a previously relayed packet was manually re-broadcast unchanged to confirm it was recognised and dropped by every node that had already seen its message ID, with no re-logging or re-relaying observed.

6.5. Discussion

What worked well. The two mitigations against the broadcast-storm problem identified in Section 2.1, a message-ID cache and a hop-count ceiling, together with the coincidental self-caching behaviour of `nextMessageId()`, were sufficient to prevent any observed relay loop on the bench mesh, with zero unbounded retransmission observed across all test bursts. Directional relevance classification matched its expected output in all five relative-position classes and across all twelve alert-type/scenario combinations tested. The fixed 24-byte packet size left substantial headroom under ESP-NOW’s 250-byte ceiling, confirming the packet design has room to grow (for instance, to add the authentication field discussed in Chapter 8) without approaching the protocol’s limit.

Limitations observed. A three-board, single-corridor bench setup is a real but modest verification: it demonstrates that the algorithms behave correctly on the inputs exercised, including two staged synthetic-heading scenarios, but it is not a field trial and says nothing about real inter-vehicle RF range, obstruction, Doppler shift, or channel contention among a much larger number of simultaneously transmitting nodes. The reliance on a single, nearly-straight recorded corridor for all naturally occurring test cases means the FRONT and REAR classes were verified under real, unmodified GPS data, while the other three classes depended on a manually staged heading override rather than a second physical route, a gap Chapter 8 identifies directly.

Relation to the problem statement. Every problem identified in Section 1.2 is addressed at least in prototype form by the verified system: the lack of an affordable inter-

vehicle alert channel is addressed by the ESP-NOW broadcast link itself; the unavailability of purpose-built V2V hardware is addressed by using commodity ESP32s; alert-noise from irrelevant directions is addressed by the directional relevance policy verified in Section 6.2; and the single-hop range limit is addressed by the hop-capped flooding verified in Section 6.3, though only up to the three-board bench ceiling actually available for testing.

6.6. Testing

No automated unit-test suite exists for this firmware at the time of writing (an explicitly tracked gap, discussed in Chapter 8); verification was instead performed as a manual functional-testing checklist against the Serial diagnostic log, summarised in Table 6.3.

Table 6.3: Manual functional verification checklist

Test ID	Scenario	Result
FT-01	Press each of the six buttons in turn; confirm the correct alert code and description are logged and broadcast	Passed
FT-02	Two boards powered on; confirm the receiver logs sender node ID, coordinates, heading, and alert description for every press	Passed
FT-03	Relay a packet back to its originator (self-echo, Section 6.3); confirm it is silently suppressed and not re-logged	Passed
FT-04	Manually inject a packet with hopCount=3; confirm it is processed/logged but not relayed further	Passed
FT-05	Stage Node 3 to a synthetic reciprocal heading; confirm classification reports OPPOSITE_APPROACHING/OPPOSITE_AWAY and the Emergency/Obstacle policies invert correctly relative to the FRONT/REAR case	Passed
FT-06	Reboot a node mid-session; confirm its duplicate-suppression cache and heading state reset (RAM-only state, no persistence) and heading is re-derived from the next two GPS fixes	Passed
FT-07	Power a node without flashing the SPIFFS data image; confirm the documented failure mode (logged CRITICAL message, constant (0,0) position) rather than a crash	Passed
FT-08	Hold a button down continuously; confirm the 250 ms debounce prevents a flood of duplicate local triggers from one physical press	Passed

Chapter 7. Conclusion

This project set out to test whether a small button panel, a license-free broadcast radio, and a modest amount of on-device heading and position math could give a following or oncoming vehicle a materially earlier hazard warning than a horn, using hardware inexpensive enough to retrofit onto an existing vehicle. The resulting system implements a complete pipeline from driver-triggered alert origination through GPS-delta heading derivation, five-way directional relevance classification, and hop-capped flooding with message-ID deduplication, all running on commodity ESP32 hardware communicating over Espressif’s connectionless ESP-NOW protocol.

Verified end-to-end on a three-board bench mesh against the project’s bundled 4.59 km recorded coordinate corridor, the system correctly classified every one of the five relative-position categories, applied the correct accept/reject verdict across all twelve alert-type/scenario combinations tested, and prevented an unbounded relay loop through the combination of hop-count capping and message-ID deduplication, including the self-echo case that a naive flood implementation would otherwise mishandle. This directly supports the project’s central design hypothesis: that directional relevance filtering and bounded flooding are sufficient to build a working, license-free V2V alerting prototype without a licensed V2X radio, a cellular subscription, or any fixed infrastructure.

Beyond the verified behaviour itself, the project’s engineering discipline is a deliverable in its own right: every load-bearing but non-obvious invariant, the originator’s own message-ID pre-caching that prevents self-echo storms, the delivery-versus-relay off-by-one in the hop-count ceiling, and the two-step application-plus-SPIFFS flashing requirement, is documented explicitly rather than left for a future contributor to rediscover the hard way. The system in its current form is a genuine working prototype, not a simulation: it broadcasts real packets between real ESP32 boards, computes real headings from a real recorded coordinate track, and correctly filters and relays real alerts. Chapter 8 discusses honestly what would be required to take it from that bench prototype to a device installable in an actual vehicle.

Chapter 8. Limitations and Future Enhancement

8.1. Current Limitations

- **No live GPS hardware.** Position and heading currently come from a pre-recorded, on-flash replay track (Section 4.3) rather than a live receiver; a node's direction is therefore only as fresh as the last three-second replay tick, and every board loaded with the same file replays an identical route.
- **No authentication or encryption.** The ESP-NOW broadcast peer is registered with `encrypt=false`; any ESP32 within range can broadcast a fabricated `AlertPacket` with an arbitrary node ID, position, or alert type. This is acceptable for a bench prototype but is unacceptable for a road deployment, where a spoofed "hard brake ahead" alert could itself provoke a dangerous reaction.
- **Manual, compile-time node identity.** `MY_NODE_ID` is a hardcoded constant edited and reflashed per board (Section 5.5) rather than derived automatically, which does not scale past a handful of hand-flashed prototype units.
- **Flood-only routing at a fixed hop ceiling.** Every relay is an unconditional broadcast bounded only by `MAX_HOPS` and the 32-entry deduplication cache (Section 5.3); a denser mesh than the three-board bench setup would multiply channel load roughly with node count, and the fixed-size cache could in principle evict a still-relevant entry under sustained multi-source traffic.
- **Serial-only driver output.** Alerts are currently visible only on a USB-serial monitor, appropriate for bench verification but unusable by a driver in a moving vehicle; there is no buzzer, LED, or in-cabin display.
- **Blocking software debounce.** `update_switches()` calls a 250 ms blocking `delay()` inline in the main loop on every detected press, which stalls the rest of `loop()`, including the next GPS/heading check, for that quarter-second.
- **Float-precision GPS in transit.** `GPSCoordinates` stores latitude/longitude as `double`, but `AlertPacket` narrows them to `float` for transmission, trading roughly two decimal digits of precision, sub-metre at these latitudes, for a smaller broadcast payload; this has not been evaluated against real receiver GPS hardware's own accuracy floor.
- **No field or RF range trial.** Hop-count and relevance logic were verified functionally on a stationary bench (Chapter 6), not against real inter-vehicle radio range, obstruction, or Doppler conditions on an actual moving-vehicle road trial.

8.2. Future Enhancements

- **Live GPS integration.** Integrate a UART GPS receiver (e.g. a NEO-6M or NEO-M8N module) behind the existing `gps_module.h` interface, which was designed with `checkAndGetGPS()`/`getLatestGPS()` as the seam for exactly this swap, with no change required to any other module.
- **Packet authentication.** Add lightweight authentication, either ESP-NOW's built-in peer encryption or a per-message HMAC computed with a pre-shared fleet key, so a receiving node can reject a forged alert before acting on it.
- **Automatic node identity.** Derive `MY_NODE_ID` from the ESP32's factory-programmed MAC address (e.g. via `WiFi.macAddress()`) instead of a hand-edited constant, removing the manual per-board flashing step.
- **Smarter relaying beyond flat flooding.** Move from unconditional flood relaying toward a policy that accounts for a node's own heading or position (for instance, relaying preferentially in the direction traffic is likely to benefit) to bound channel load as node density grows beyond a handful of vehicles.
- **Driver-facing output.** Add an in-cabin buzzer and small display, or a companion smartphone app over Bluetooth Low Energy, so alerts are usable while driving instead of requiring a USB-serial monitor.
- **Non-blocking debounce.** Replace the blocking `delay(250)` debounce with a timestamp-based, non-blocking implementation so button handling never stalls the GPS/heading loop.
- **Persist configuration and recent messages.** Use the `nvs` partition already reserved in the flash layout (Table 1.2) but currently unused, so a reboot does not silently reset the deduplication cache or require re-entering node configuration.
- **Field and range trial.** Conduct a moving-vehicle field trial to validate the currently bench-only hop-count and directional-relevance behaviour under real RF propagation, obstruction, and multi-vehicle contention.

Bibliography

- [1] J. B. Kenney, “Dedicated Short-Range Communications (DSRC) Standards in the United States,” *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.
- [2] 3GPP, “Study on LTE Support for V2X Services,” 3GPP TR 22.885, 2016.
- [3] S. Ni, Y. Tseng, Y. Chen, and J. Sheu, “The Broadcast Storm Problem in a Mobile Ad Hoc Network,” in *Proceedings of ACM MobiCom*, 1999.
- [4] C. Perkins, E. Belding-Royer, and S. Das, “Ad hoc On-Demand Distance Vector (AODV) Routing,” IETF RFC 3561, 2003.
- [5] J. Blum, A. Eskandarian, and L. Hoffman, “Challenges of Intervehicle Ad Hoc Networks,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 5, no. 4, pp. 347–351, 2004.
- [6] T. ElBatt *et al.*, “Cooperative Collision Warning Using Dedicated Short Range Wireless Communications,” in *Proceedings of ACM VANET*, 2006.
- [7] Espressif Systems, “ESP-NOW User Guide,” Espressif Systems Documentation, accessed 2026.
- [8] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems, 2026.
- [9] Arduino, “Arduino Core for ESP32,” open-source software repository, accessed 2026.
- [10] PlatformIO, “PlatformIO Documentation,” <https://docs.platformio.org>, accessed 2026.

Appendix 1. Firmware Configuration Reference

Table 1.1 lists every compile-time constant that controls the firmware’s runtime behaviour. Table 1.2 lists the flash partition layout each board is built with.

Table 1.1: Firmware configuration constants

Constant	Default	Effect
MY_NODE_ID (main.cpp)	1	Node identity; must be set uniquely per board before flashing
MAX_HOPS (multihop_relay.h)	3	Relay ceiling; hop counts up to this value are relayed onward
MESSAGE_CACHE_SIZE (multihop_relay.cpp)	32	Depth of the duplicate-suppression ring buffer
LOCATION_UPDATE_INTERVAL (gps_module.cpp)	3000	Milliseconds between GPS replay ticks
Direction-check interval (main.cpp)	3000	Milliseconds between heading recomputation in loop()
Button debounce delay (switch_input.cpp)	250	Milliseconds a button read is blocked after a detected press
headingToleranceDegrees (directional_awareness.h)	60.0	Same-lane/opposite-lane classification tolerance
buttonPins[] (switch_input.cpp)	6 pins	GPIO 13, 12, 14, 27, 26, 25 for the six alert buttons, in that order
monitor_speed (platformio.ini)	115200	Serial diagnostic baud rate

Table 1.2: Flash partition layout (huge_app.csv, 4 MB flash)

Name	Type	Offset	Size	Used for
nvs	data	0x9000	20 KB	Reserved, not yet used by the firmware
otadata	data	0xE000	8 KB	Reserved, no OTA update path implemented
app0	app	0x10000	3 MB	Application firmware image
spiffs	data	0x310000	896 KB	Recorded GPS replay track (location_data.txt)
coredump	data	0x3F0000	64 KB	Crash diagnostics

Appendix 2. Sample Serial Monitor Log

The excerpt below is representative of the diagnostic output produced on the bench setup of Chapter 6: Node 1 (identity 1) originates an Obstacle alert, which Node 2 (identity 2) receives, classifies, logs, and relays.

```
--- Node 1 ---
Initializing V2V Node #1...
Location replay ready. Reading /location_data.txt every 3 seconds.
[Direction] Updated vehicle heading to 93.5 using GPS delta.
System initialization complete. Monitoring channel...

[Local Trigger] Generating new alert...
Switch 5 pressed! Triggering alert...
[TX] New messageId=16777316, hopCount=0
[Local Trigger] Alert Type: 5 (Potholes/Obstacle ahead)
[Local Trigger] Sending heading: 93.5
Latitude: 27.675250207803, Longitude: 85.352358994142
-> Packet Transmit Status: SUCCESS
[Local Trigger] Alert packet broadcast queued.

--- Node 2 ---
[RX] messageId=16777316 from node #1, hopCount=0.
[RELEVANCE] messageId=16777316 is relevant; processing alert.

===== [ NEW ESP-NOW PACKET RECEIVED ] =====
Sender Node ID: 1
Message Sequence ID: 16777316
Coordinates    : Lat: 27.675250, Lng: 85.352359
Sender Heading : 93.500000
Receiver Heading: 93.500000
Current Hop    : 0
Alert Type Code: 5
Alert Message  : Potholes/Obstacle ahead
=====

[RELAY] messageId=16777316 will be relayed from hopCount=0.
[RELAY] Forwarding messageId=16777316 with hopCount=1.
-> Packet Transmit Status: SUCCESS
```

Listing 2.1: Representative Serial monitor excerpt (two boards, 115200 baud)